DEVELOPMENT OPERATIONS

2025

CS 328

Submitted by**: AHMED AMJAD, SHAMINA DURRANI**

Registration No.: **2022063,2022543**

Faculty: **FCSE**

"On my honor, as student of Ghulam Ishaq Khan Institute of Engineering Sciences and Technology, I have neither given nor received unauthorized assistance on this academic work."

Submitted to:

Lecturer Ahmad Nawz

Date: 16 April, 2025

Project Proposal: URL Shortener with DevOps Automation

1. Project Overview

We propose building a **lightweight URL shortener service** (similar to Bit.ly) with a focus on **DevOps automation**. The project will include:

- A **Flask-based REST API** to shorten URLs and track redirects.
- **Full CI/CD pipeline** (GitHub Actions → Docker → AWS EC2).
- **Infrastructure as Code (IaC)** using Terraform.
- **Monitoring** with Prometheus and Grafana.

This project is ideal for learning **end-to-end DevOps practices** while keeping development simple.

2. Objectives

Goal	Outcome
Functional URL shortener	Users can submit a long URL and get a shortened link.
Automated deployments	Code changes auto-deploy via CI/CD (GitHub Actions + Docker).
Infrastructure automation	Terraform provisions an EC2 instance with one command.
Basic monitoring	Grafana dashboard shows URL creation rate and redirect hits.

3. Technical Stack

Component	Technology	Purpose
Backend	Python (Flask)	Lightweight API for URL shortening.
Database	SQLite	Simple file-based storage (no external DB setup).
CI/CD	GitHub Actions	Automate testing, Docker builds, and deployment.
Container	Docker	Package the app for consistent deployments.
Infrastructure	Terraform (AWS EC2)	Automate cloud server provisioning.
Monitoring	Prometheus + Grafana	Track metrics (URLs created, redirects).

4. Implementation Plan

Phase 1: Core API Development (Week 1)

- Develop Flask endpoints:
    - POST /shorten → Create short URL.
    - GET /<code> → Redirect to original URL.
- Implement SQLite database for storage.
- Add basic error handling.

Phase 2: DevOps Automation (Week 2)

- **Dockerize** the Flask app.
- Set up **GitHub Actions CI/CD**:
    - Run tests on push.

    - Build and push Docker image to Docker Hub.
- Write **Terraform scripts** to deploy to AWS EC2.

Phase 3: Monitoring & Analytics (Week 3)

- Integrate **Prometheus** to collect metrics (hits, response times).
- Set up a **Grafana dashboard** for visualization.
- (Optional) Add logging for failed redirects.

4. Expected Deliverables

Deliverable	Description
Flask API	Fully functional URL shortener (2 endpoints).
Docker Image	Ready-to-deploy containerized app.
CI/CD Pipeline	GitHub Actions workflow for auto-deployment.
Terraform Scripts	IaC to provision AWS EC2 instance.
Monitoring Dashboard	Grafana visualization of URL stats.

5. Conclusion

This project provides a **practical, DevOps-focused** approach to building a URL shortener while covering key modern deployment practices. It's **low-complexity but high learning value**, making it ideal for engineers looking to strengthen their automation skills.

Next Steps:

- Approve proposal.
- Set up repository + initial scaffolding.
- Begin Phase 1 (API development).